

OPERATING SYSTEMS

UE25CS242A

Operating-System Services and Process Concept

Dr. Anitha K
Department of Computer Science & Engineering

Acknowledgement, Prof. Pavan A C

Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources:



1. "Operating System Concepts", Abraham Silberschatz, Peter Baer Galvin, Greg Gagne 10th Edition, John Wiley & Sons, Global Edition, 2023.
2. "Advanced Programming in the Unix Environment", Richard Stevens and Stephen A Rago, Pearson, 3rd edition, 2017.
3. Some conceptual text and diagram from "Operating Systems, Internals and Design Principles", William Stallings, 9th Edition, Pearson, 2018

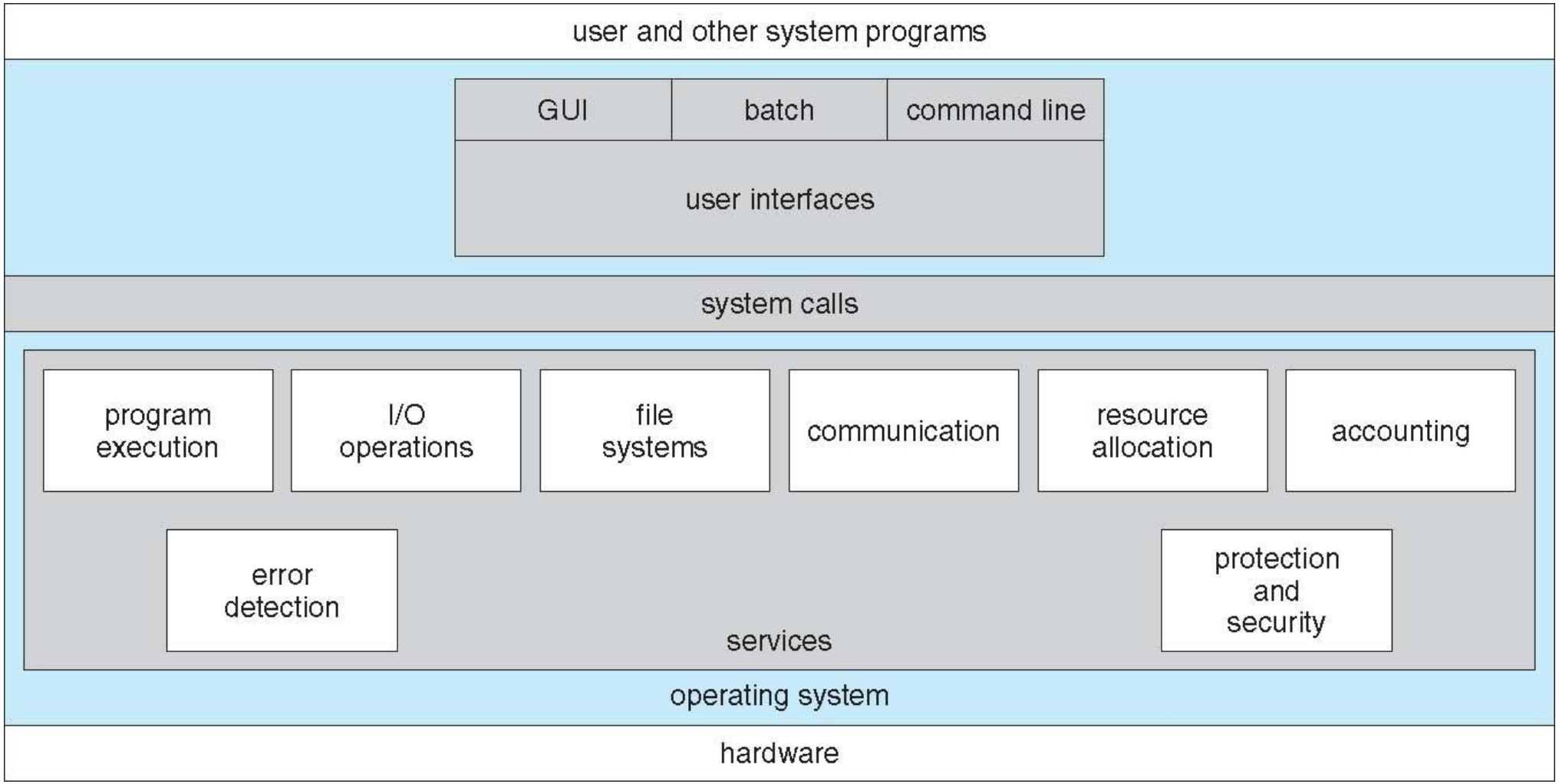
OPERATING SYSTEMS

Operating-System Services and Process Concept

Dr. Anitha K
Department of Computer Science and Engineering

OPERATING SYSTEMS

A View of Operating System Services



© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS

Services

- Operating systems provide an **environment for execution of programs and services to programs and users**
- Set of operating-system services provides functions that are helpful to the user:
  - **User interface** - Almost all operating systems have a user interface (**UI**).
    - **Command-Line (CLI) -** Command interpreters (shells)
    - **Graphics User Interface (GUI)**
    - **Batch**
  - **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating an error)
  - **I/O operations** - A running program may require I/O, which may involve a file or an I/O

Services

- **File-system manipulation -** The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.
- **Communications** – Processes may exchange information, on the same computer or between computers over a network
  - Communications may be via shared memory or through message passing (packets moved by the OS)

Services



- **Error detection** – OS needs to be **constantly aware of possible errors**
  - May occur in the **CPU and memory hardware, in I/O devices, or in a user program.**
  - For each type of error, OS should take the **appropriate action to ensure correct and consistent computing.**
  - **Debugging** facilities can greatly **enhance the user's and programmer's abilities to efficiently use the system**.

Services

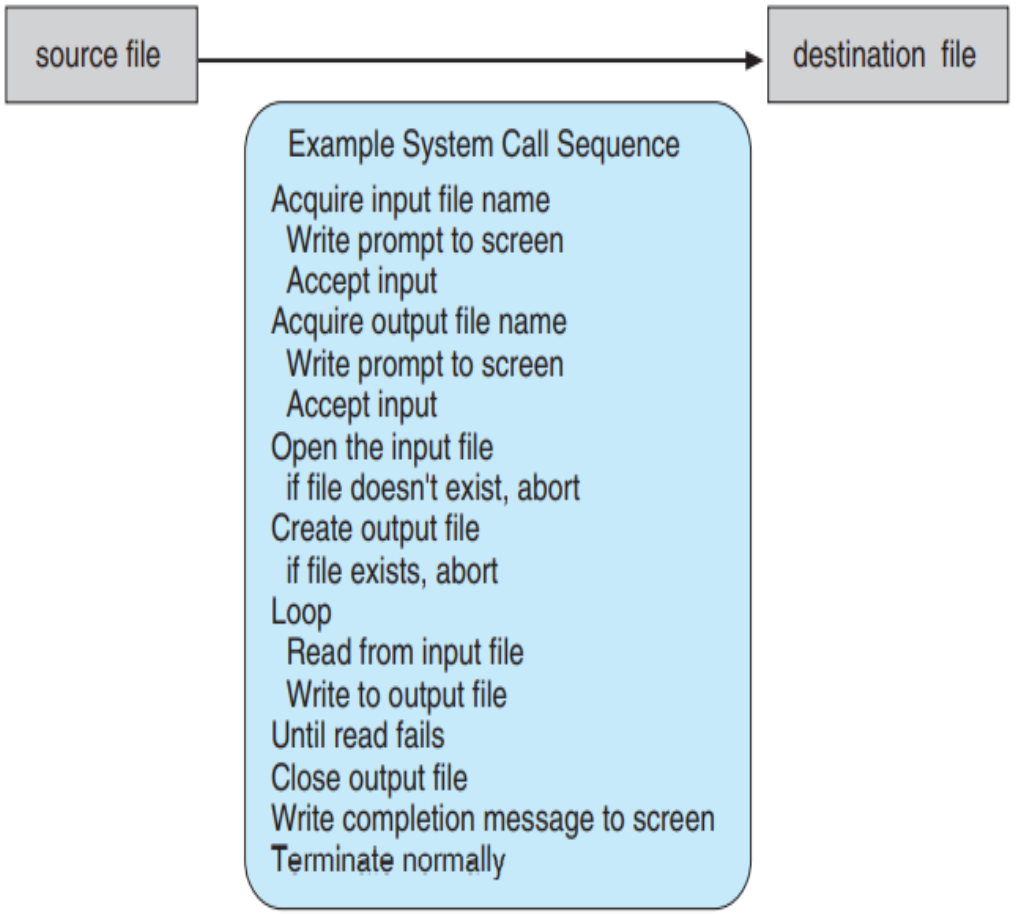
- Another set of OS functions exists for ensuring the efficient operation of the system itself via **resource sharing**
  - **Resource allocation –** When multiple users or multiple jobs are running concurrently, resources must be allocated to each of them.
    - Many types of resources - CPU cycles, main memory, file storage, I/O devices.
  - **Accounting -** To keep track of which users **use** how much and **what kinds of computer resources.**

Services

- **Protection and security -** The owners of information stored in a multiuser or networked computer system may want **to control use of that information. Concurrent processes should not interfere with each other**.
  - **Protection** involves ensuring that all **access to system resources is controlled.**
  - **Security** of the system from outsiders **requires user authentication** ,which extends to defending external I/O devices

System calls

- **System calls** provide an **interface to the services** made available by an operating system.
- These calls are generally **available as routines** written in a higher level programming language or assembly-language.
- Example: Sequence of system calls used



Types of System calls

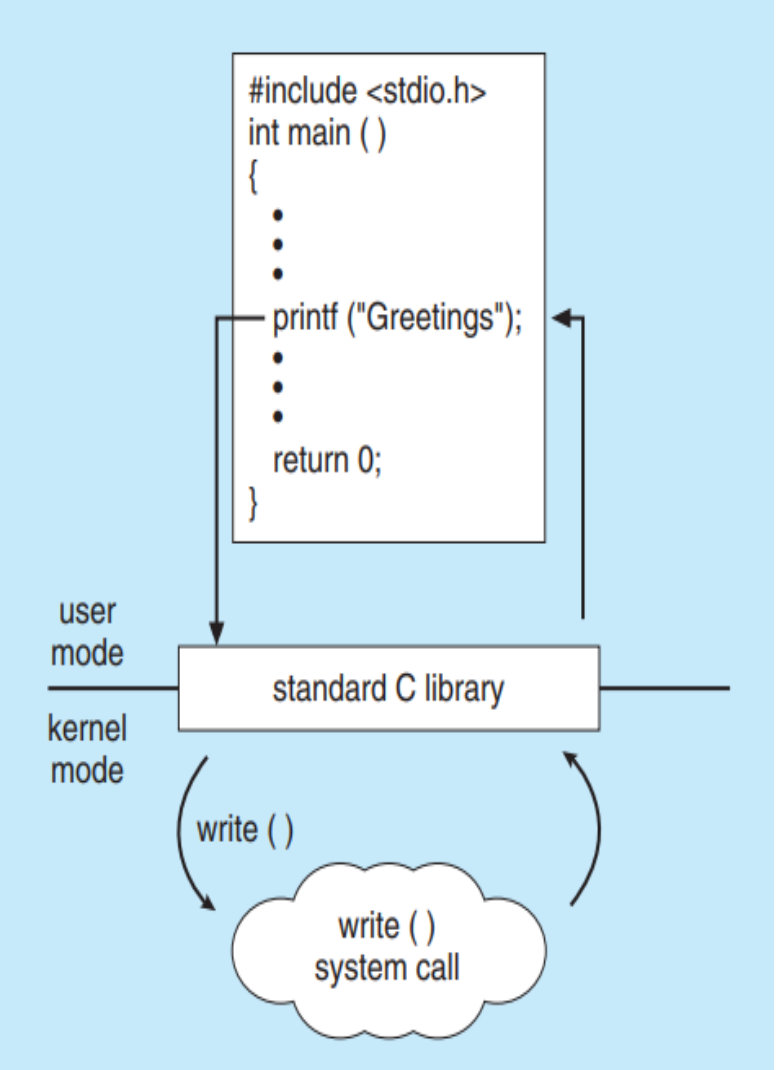
System calls can be grouped roughly into six major categories:

1. Process control
2. File manipulation
3. Device manipulation
4. Information maintena
5. Communications
6. Protection

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Example of a system call

- A C program invokes the printf() statement.
- The C library intercepts this call and invokes the necessary system call **write()** in the operating system.
- The C library takes the value returned by write() and passes it back to the user program.

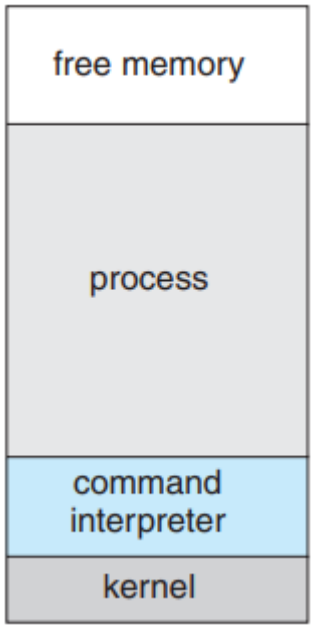


PES UNIVERSITY
CELEBRATING 50 YEARS

OPERATING SYSTEMS

Process Concept

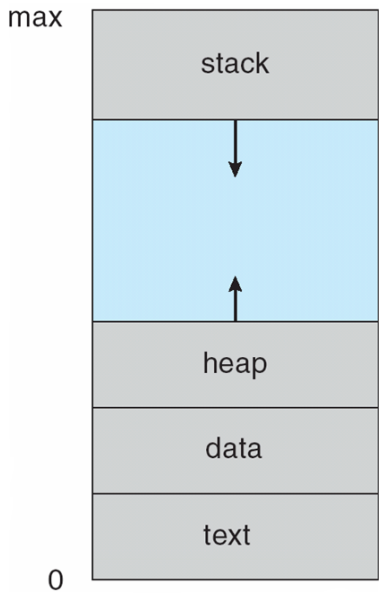
- An operating system executes a variety of programs:
  - Batch system – **jobs**
  - Time-shared systems – **user programs** or **tasks**
- Textbook uses the terms ***job*** and ***process*** almost interchangeably
- **Process** – a program in execution; process execution must progress in sequential fashion
- Program is ***passive*** entity stored on disk (**executable file**), process is ***active***
  - Program becomes process when executable file loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc
- One program can be several processes
  - Consider multiple users executing the same program



PES UNIVERSITY
CELEBRATING 50 YEARS

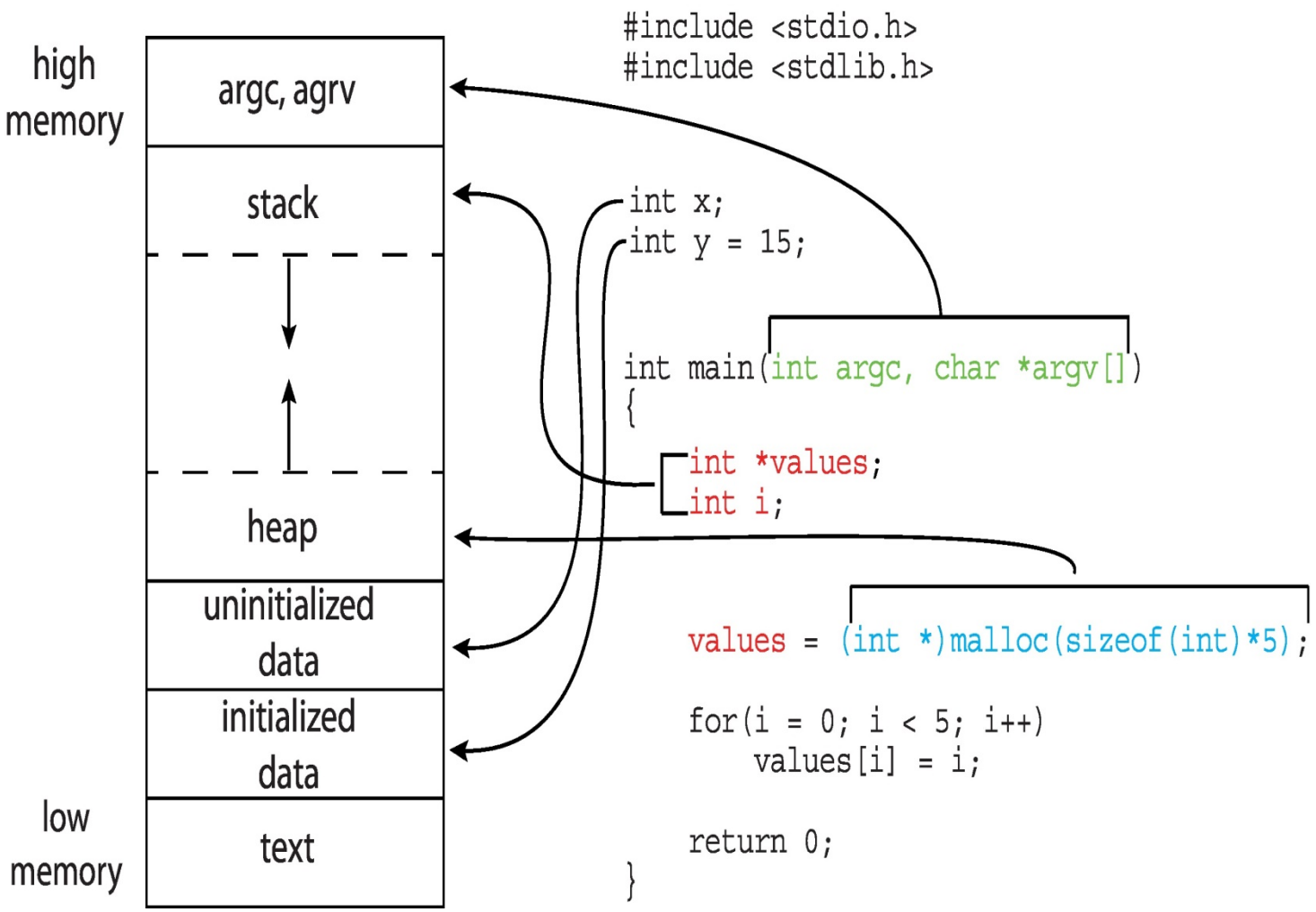
Structure of a process in memory

- The program code, also called **text section.**
  - Includes current activity including **program counter**, processor registers
- **Stack** containing temporary data
  - Function parameters, return addresses, local variables
- **Data section** containing global variables
- **Heap** containing memory dynamically allocated during run time



Layout of a process in memory.

Memory Layout of a C Program



OPERATING SYSTEMS

Process State

As a process executes, it changes **state**

- **New**: The process is being created
- **Running**: Instructions are being executed
- **Waiting**: The process is waiting for some event to occur
- **Ready**: The process is waiting to be assigned to a processor
- **Terminated**: The process has finished execution

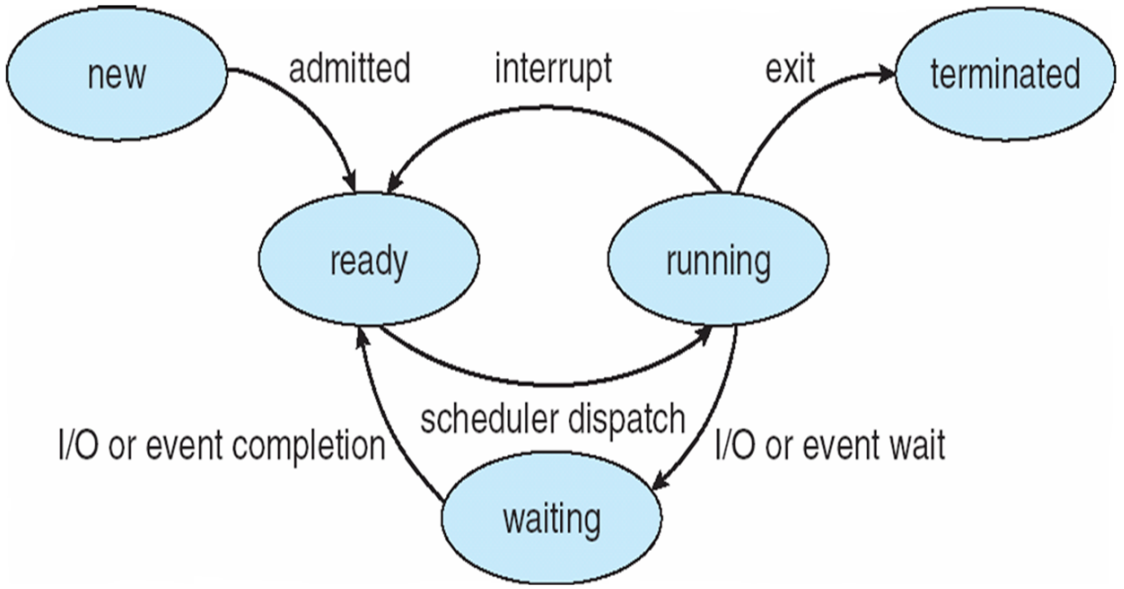


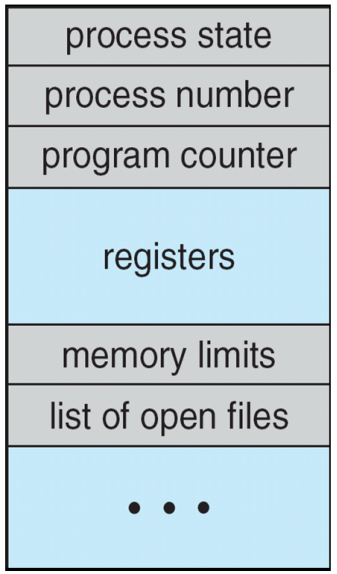
Diagram of process state.

OPERATING SYSTEMS

Process Control Block (PCB)

Each process is represented in the operating system by a Process Control Block (also called **task control block**)

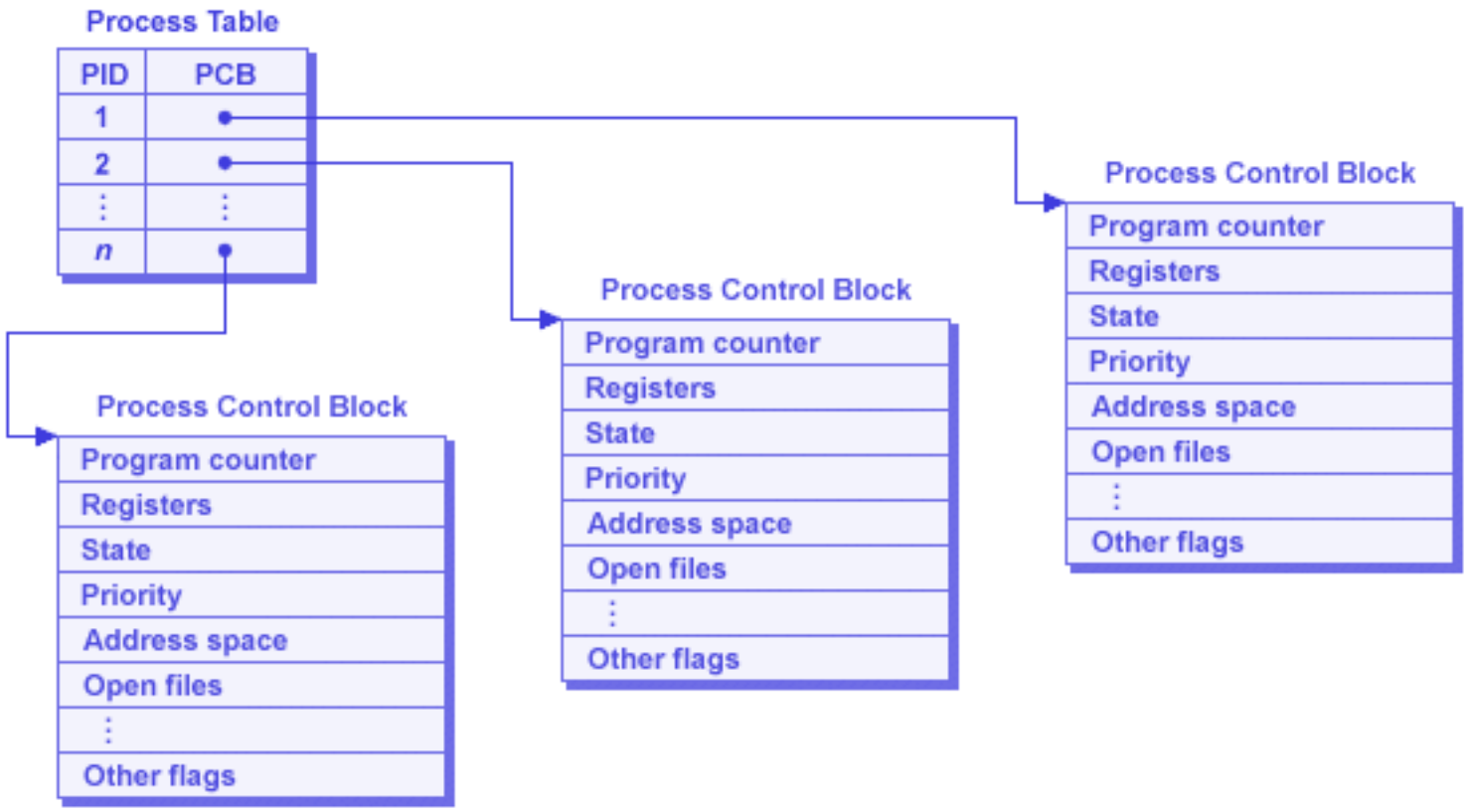
- Process state – running, waiting, etc
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list



Process control block (PCB).

OPERATING SYSTEMS

Process Control Block (PCB)



PES UNIVERSITY
CELEBRATING 50 YEARS

OPERATING SYSTEMS

Process Scheduling



- Maximize CPU use, quickly switch processes onto CPU for time sharing
- **Process scheduler** selects among available processes for next execution on CPU
- Maintains **scheduling queues** of processes
  - **Job queue** – set of all processes in the system
  - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
  - **Device queues** – set of processes waiting for an I/O device
  - Processes migrate among the various queues

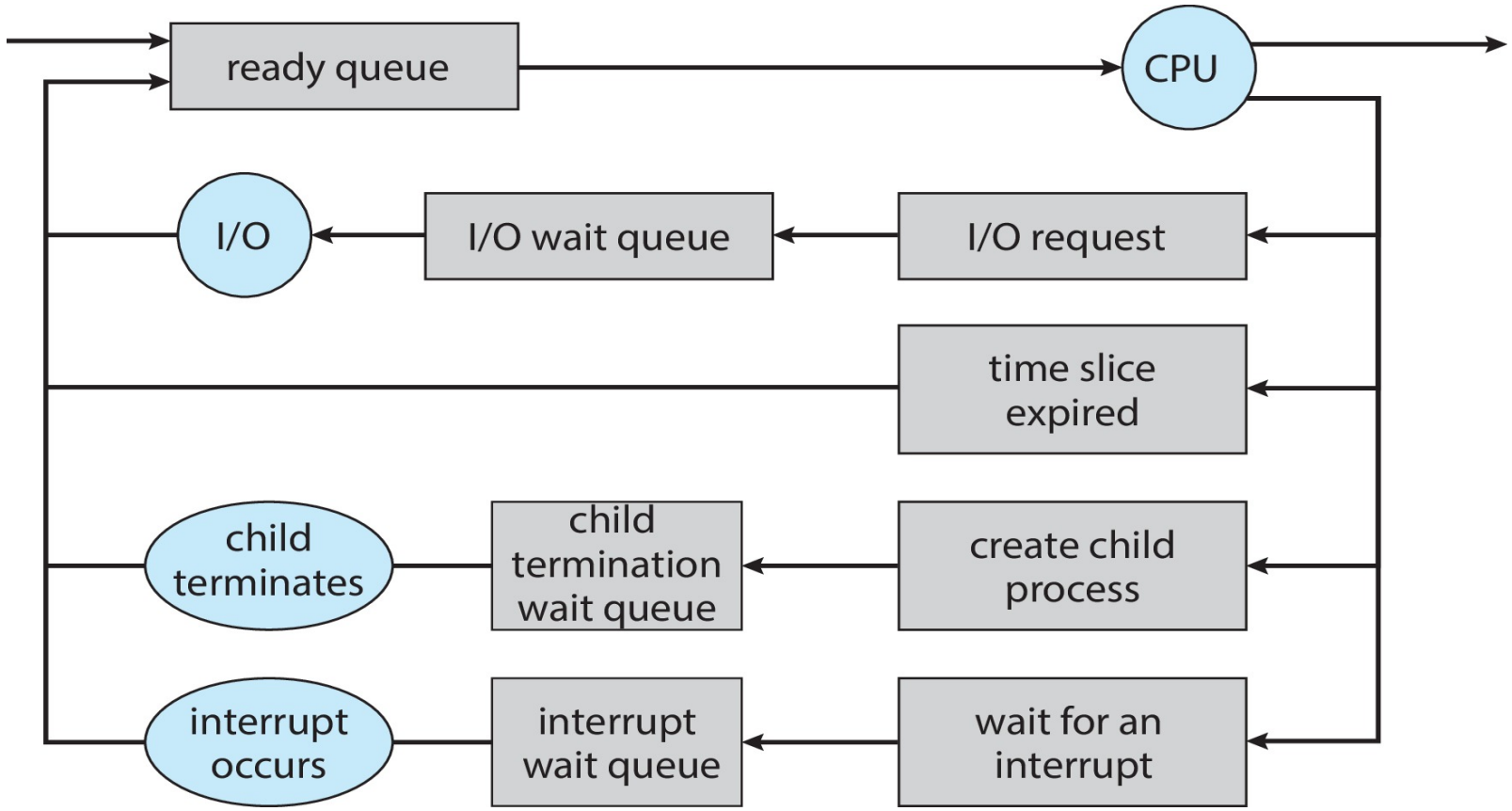
Schedulers



- Processes can be described as either:
  - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
  - **CPU-bound process** – spends more time doing computations; few very long CPU bursts
- Long-term scheduler strives for good ***process mix***

OPERATING SYSTEMS

Representation of Process Scheduling



Queueing-diagram representation of process scheduling

© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS

CPU switch from process to process

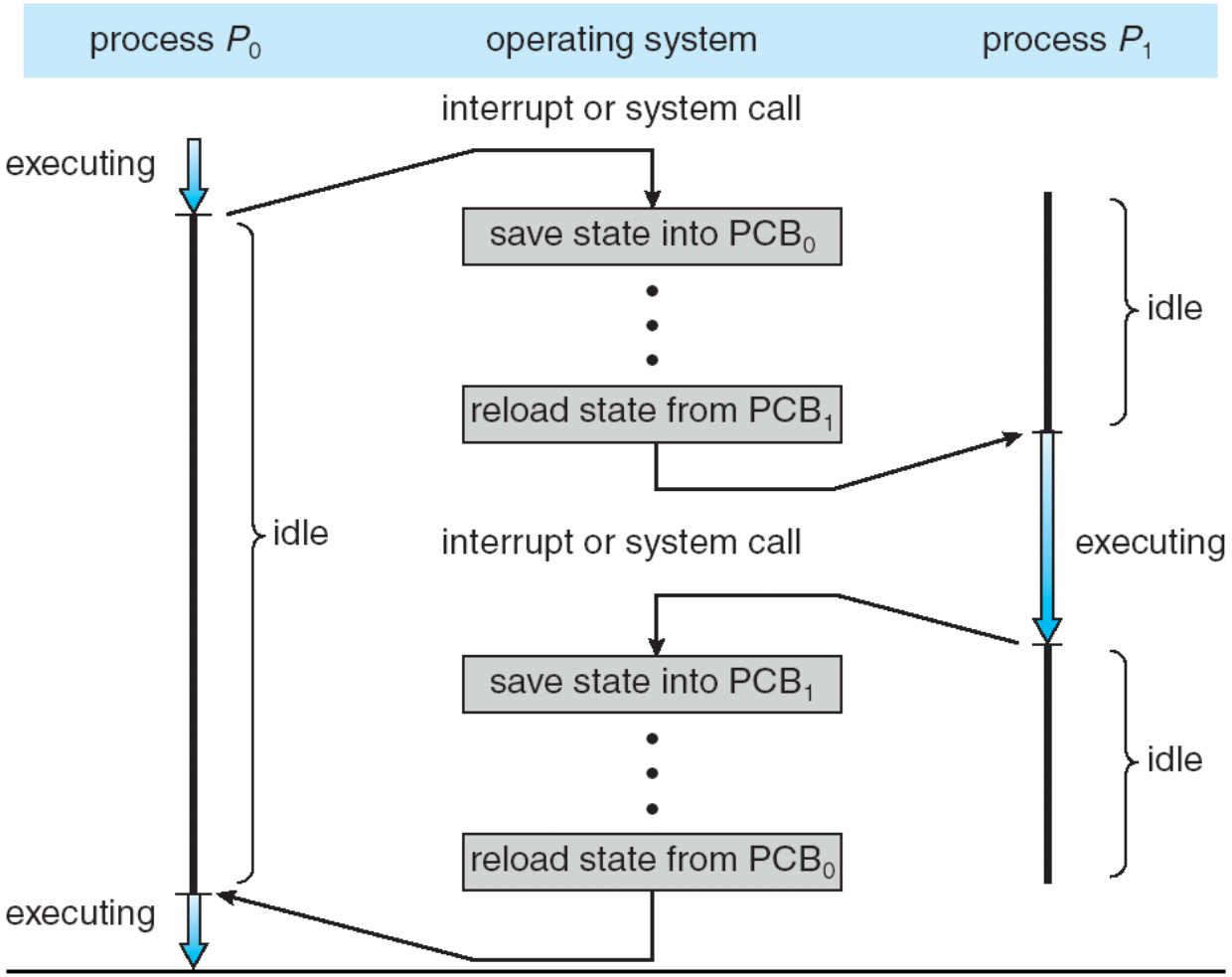


Diagram showing context switch from process to process.

OPERATING SYSTEMS

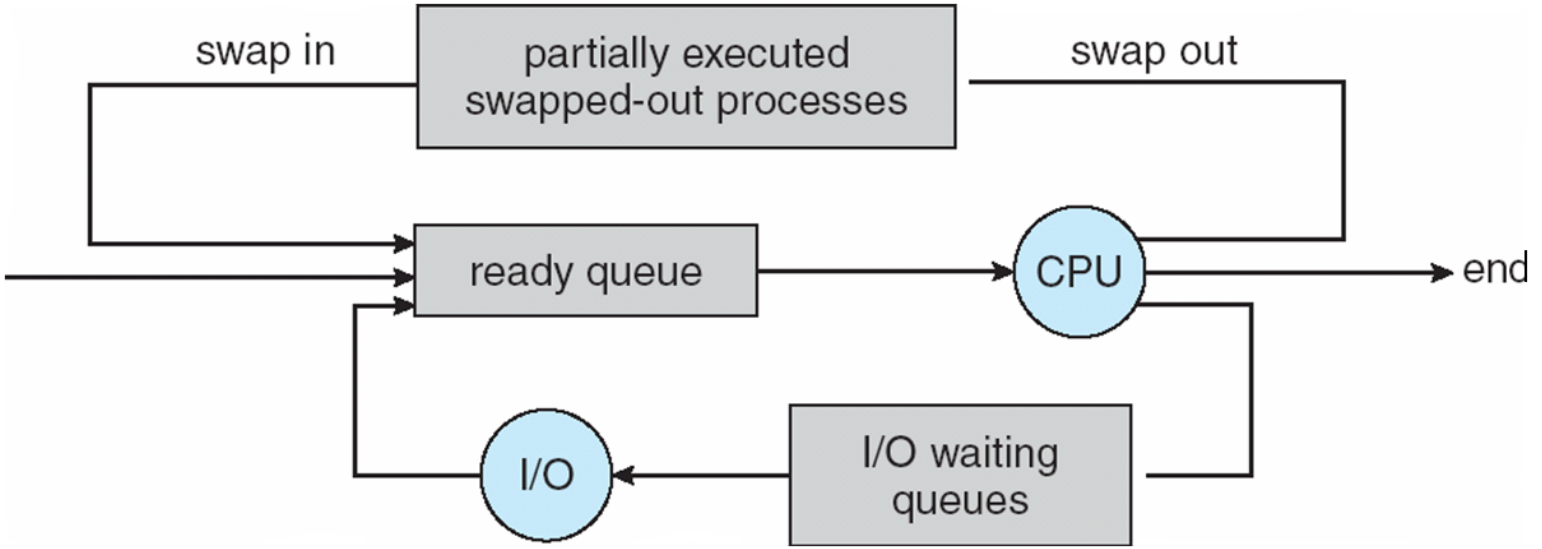
Schedulers



- **Short-term scheduler** (or **CPU scheduler**) – selects which process should be executed next and allocates CPU
  - Sometimes the only scheduler in a system
  - Short-term scheduler is invoked frequently (milliseconds) ⇒ (must be fast)
- **Long-term scheduler** (or **job scheduler**) – selects which processes should be brought into the ready queue
  - Long-term scheduler is invoked infrequently (seconds, minutes) ⇒ (may be slow)
  - The long-term scheduler controls the **degree of multiprogramming**

Representation of Process Scheduling

- **Medium-term scheduler** can be added if degree of multiple programming needs to decrease
  - Remove process from memory, store on disk, bring back in from disk to continue execution: **swapping**



© copyright Silberschatz, Galvin and Gagne, 2013

Context Switch

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is overhead; the system does no useful work while switching
  - The more complex the OS and the PCB ➔ the longer the context switch



THANK YOU

Dr. Anitha K
Department of Computer Science & Engineeri

anithak@pes.edu